ARE SELF-EVOLVING AGENT SKILLS LEARNING SKILLS?

Anonymous Submission

ABSTRACT

We introduce a lifecycle audit for determining whether score gains from self-evolving agent skills represent retained skill improvement. A post-revision score alone is insufficient because optimizer variation, validation selection, execution randomness, and verifier choice can improve the endpoint without creating a durable artifact effect. We combine repeated evaluations, controlled task variations, and lifecycle interventions to examine whether improvements from skill evolution persist and transfer, and to trace their origins to feedback, revision, selection, skill content, and evaluation. The audit separates five evidential claims: selected gain, retained artifact effect, bounded transfer, evolution-pathway dependence, and measurement dependence. This claim-grounded framework lets self-evolving systems report what their experiments establish without equating a better benchmark score with reusable skill learning.

***Index Terms*—** Agent skills, self-evolution, lifecycle audit, skill evaluation, generalization

1. INTRODUCTION

Persistent agent skills offer an appealing route to continual improvement. Instead of updating model parameters, an agent can revise an external artifact that stores reusable instructions, scripts, references, templates, or other supporting resources. Recent systems generate, repair, select, and retain such artifacts from execution traces and evaluator feedback, often reporting higher post-evolution benchmark scores as evidence that the agent has learned a better skill [1–4].

A higher score is not, by itself, an identified skill effect. The selected run may be unusually favorable; the optimizer may produce variable candidates; a validation gate may retain a narrow patch; repeated execution may eventually succeed without a durable behavioral change; and a verifier may reward only part of the intended behavior. These possibilities do not imply that self-evolution is ineffective. They imply that an endpoint score alone cannot locate where the improvement resides.

We therefore ask two connected questions. When self-evolution raises the benchmark score, does the revised skill itself retain a behavioral improvement under fresh deployment and controlled task variation? Regardless of whether it does, which parts of the evolution-and-evaluation lifecycle produced the observed gain?

We organize the evidence as a diagnosis of utility, behavioral scope, and gain attribution. Standard-task evaluations of SkillEvolver, SPARK, and CoEvoSkills first establish the observed utility of completed artifacts across two runs. Controlled task changes then reveal different scopes of usefulness, and artifact and trace inspection examines the decisions preserved during reuse. This motivates the next diagnostic step: locating an improvement in the artifact's evolution history. Complementary studies of SkillRevise, SkillHone, and SkillOpt use retention tests and lifecycle interventions to examine trace-conditioned revision, history-based iterative honing, and validation-gated editing. Each study addresses a specific part of the identification question.

Our audit follows the lifecycle from feedback to revision, selection or adoption, frozen artifact deployment, and measurement. We separate historical success from fresh retention, final-skill utility from actual parent-to-final revision effects, and same-task retention from controlled transfer. We then intervene on feedback, optimizer sampling, gate provenance, artifact components, and verifier choice. A short information-integrity check bounds direct answer or oracle exposure as an alternative explanation; it is not the paper's organizing hypothesis.

The resulting picture is deliberately more qualified than either *self-evolution works* or *self-evolution only exploits the evaluator*. Revised artifacts can cause real behavioral improvements, but the demonstrated effects are often narrow. Single-run or selected gains need not survive fresh deployment. Useful traces and feedback must be converted into an effective edit, adopted into the persistent artifact, and retained behaviorally. Finally, a better score can reflect compatibility with a measurement surface rather than broader task behavior.

Our central contribution is a claim-grounded lifecycle audit: each reported gain is matched to the intervention required to support it and to the strongest conclusion that intervention permits. We make three contributions:

- **Identification framework.** We separate a reported gain from retention, behavioral scope, evolution-and-selection pathway, and measurement dependence.
- **Lifecycle audit.** We apply matched interventions across three representative skill-evolution mechanisms without turning heterogeneous studies into a method leaderboard.
- **Reporting protocol.** We specify the lineage, contrasts, and artifacts required to support each claim level, from selected gain to bounded transfer.

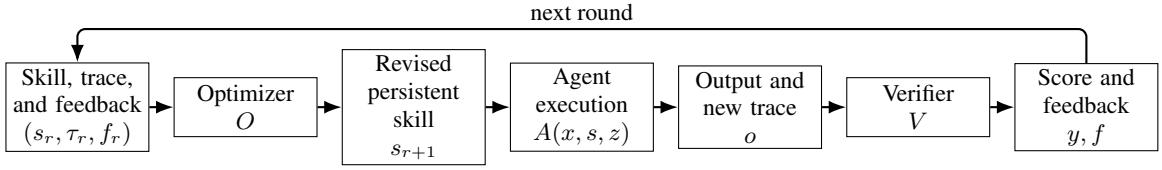


Fig. 1. Minimal self-evolution lifecycle. The update operator includes any method-native validation, selection, merge, or adoption before the revised skill is retained.

2. PROBLEM FORMULATION

2.1. Skill Revision and Evaluation

At revision round r, an optimizer updates the current persistent skill artifact:

$$s_{r+1} = O(s_r, \tau_r, f_r), \tag{1}$$

where s_r is the current skill, τ_r is an execution trace, and f_r is feedback that may include a verifier result, a gold answer, a diagnosis, or evaluator feedback. The operator O denotes the method's complete update pathway, including its native merge, validation, selection, or adoption mechanism. A generated candidate that is rejected or never adopted is not s_{r+1}.

The agent then executes task x with the frozen skill:

$$o_r = A(x, s_r, z_r), \tag{2}$$

where z_r represents model sampling, tool use, and environment state, and o_r is the resulting output artifact or state change. A verifier maps the output to a score:

$$y_r = V(x, o_r). \tag{3}$$

2.2. The Identification Question

Most evaluations observe:

$$y_{r+1} > y_r. \tag{4}$$

This inequality describes two evaluations; it does not identify what the persistent skill learned. The revised artifact may encode a retained behavioral improvement, the selected score may depend on optimizer or execution variation, or the output may better match the verifier's measurement surface. Infrastructure-invalid observations, such as a verifier that never reaches its semantic assertions, are neither successes nor failure mechanisms and must be repaired or excluded.

Accordingly, we separate two questions:

1. **Effect existence and scope:** Does the revised skill retain a behavioral improvement under fresh deployment, and how far does that effect extend under controlled task variation?
2. **Effect attribution:** Which feedback, revision, selection, artifact, execution, and measurement mechanisms produced the observed score change?

The following experiments address these questions in turn.

3. DIAGNOSING THE SCOPE OF SKILL UTILITY

The first diagnostic question is where a completed skill is useful. A positive source-task result can arise from content that applies across instances or from a procedure fitted to the original task. We distinguish these possibilities by first examining paired utility in two standard-task runs, then deploying frozen skills under three forms of task variation. Inspection of skill content and execution traces connects the resulting utility profiles to decisions made during deployment.

3.1. Experimental Setup

We evaluate SkillEvolver [5], SPARK [6], and CoEvoSkills [7] on SkillsBench [8], with two runs per method. Each run pairs an execution using the generated or evolved skill with a no-skill execution of the same task. Both runs of each method use the same task set: 81 tasks for SkillEvolver, 84 for SPARK, and 85 for CoEvoSkills.

We report mean verifier reward on a 0–100 scale and the mean within-task difference between the skill and no-skill conditions. For a run with N tasks, the paired gain is

$$\Delta = \frac{100}{N} \sum_{i=1}^{N} \left(r_i^{\text{skill}} - r_i^{\text{no-skill}} \right). \tag{5}$$

We also count tasks with positive, negative, and zero differences.

3.2. Source-Task Gains Recur across Runs

All three methods have positive mean paired gains in both runs (Table 1). SkillEvolver gains 25.61 and 25.02 points, CoEvoSkills gains 10.24 and 9.76, and SPARK gains 1.84 and 6.45. SkillEvolver and CoEvoSkills retain similar gains despite substantial changes in their no-skill scores. Their benefit is therefore more consistent across runs than their absolute skill-condition scores suggest.

The task-level results also differ. SkillEvolver has at least four times as many improvements as regressions in each run; CoEvoSkills improves 17 tasks in each run, with nine and ten regressions. SPARK's first run is nearly balanced, while its second has 13 improvements and five regressions. These results establish recurring source-task utility. We next ask whether the same frozen skills remain useful after the task changes.

Method	Run	Tasks	No skill	Skill	Δ	Improved	Regressed	Tied
SkillEvolver	1	81	30.20	55.81	+25.61	27	6	48
SkillEvolver	2	81	37.32	62.34	+25.02	26	3	52
SPARK	1	84	17.82	19.66	+1.84	9	8	67
SPARK	2	84	14.37	20.81	+6.45	13	5	66
CoEvoSkills	1	85	42.21	52.44	+10.24	17	9	59
CoEvoSkills	2	85	23.54	33.31	+9.76	17	10	58

Table 1. Two runs per method on SkillsBench. Scores are mean verifier rewards multiplied by 100; Δ is the mean paired gain in percentage points. Each method uses an identical task set across its two runs.

3.3. Task Changes Separate Different Scopes of Utility

We apply each source-task skill to controlled variants without further evolution. Input-representation changes (P1) alter names, column arrangements, or schemas; task-instance changes (P2) replace the data or instance structure; coupled-requirement changes (P3) alter or add requirements that must be satisfied together. These variations test whether a frozen skill remains useful when the executor must reinterpret inputs, recompute a solution, or accommodate new requirements.

The complete record contains 222 variants and 886 condition-level results across no skill and the three methods. Table 2 summarizes historical and recovery evaluations for 222 SkillEvolver variants and 221 each for SPARK and CoEvoSkills, pairing each skill condition with the no-skill result on the same variant.

Source-task improvement does not imply useful generalization. All three methods improve mean reward on the source tasks in both runs, but their variant results diverge. SkillEvolver retains a positive mean gain of 11.94 points, SPARK is close to neutral at $+0.65$, and CoEvoSkills falls below the no-skill baseline at -5.01. SkillEvolver also improves more variants than it regresses (47 versus 20), whereas SPARK and CoEvoSkills show the reverse. Thus, the source-task gains of the latter two methods do not translate into a consistent advantage on changed tasks.

Replacing the instance produces the largest divergence. SkillEvolver remains positive under all three changes, with its largest gain on P2 ($+14.56$). P2 is also where SPARK and CoEvoSkills decline most (-5.23 and -9.51). SPARK retains positive averages under representation and requirement changes, but loses its advantage when the underlying instance changes. SkillEvolver's positive results across P1–P3 indicate a broader observed scope of skill utility. CoEvoSkills' negative averages across all three changes show that its source-task benefit is particularly sensitive to reuse conditions.

A. Overall utility on task variants

Method	Variants	No skill	Skill	Δ	Improved	Regressed	Tied
SkillEvolver	222	41.87	53.81	$+11.94$	47	20	155
SPARK	221	41.61	42.25	$+0.65$	32	37	152
CoEvoSkills	221	41.61	36.60	-5.01	26	41	154

B. Utility by type of task change

Method	P1: Representation		P2: Instance		P3: Requirements	
	N	Δ	N	Δ	N	Δ
SkillEvolver	77	$+11.85$	68	$+14.56$	77	$+9.72$
SPARK	77	$+2.00$	68	-5.23	76	$+4.53$
CoEvoSkills	77	-3.59	68	-9.51	76	-2.42

Table 2. Skill utility under task variation: descriptive summary of historical and recovery evaluations. Panel A reports overall outcomes; Panel B separates representation, instance, and requirement changes. Each method is paired with its own same-variant no-skill baseline. Scores and within-variant differences use the scale in Table 1.

3.4. What the Evolved Skills Preserve

Successful skills retain guidance for choosing and checking. In exoplanet detection, the SkillEvolver skill supports successful detection after input columns are rearranged (P1) and after the task requires two signals (P3). The CoEvoSkills skill fails both variants: its use leads to stellar modulation being reported as the target signal, and a signal and its harmonic being treated as two separate signals. The SkillEvolver skill explicitly organizes candidate discovery, signal diagnosis, and refinement, including checks for misleading peaks. This case suggests that preserving criteria for accepting a result can be more useful under task changes than retaining a search routine alone. Both skills fail the replacement-instance P2 variant, so the successful checks address only part of this task family's variation.

A similar distinction appears in scheduling. The SkillEvolver and SPARK skills succeed on all three variants; the CoEvoSkills skill succeeds only on P3. In P1, using the CoEvoSkills skill yields an otherwise valid schedule that fails the requirement to place each operation at its earliest feasible time. The SkillEvolver skill explicitly includes this local-minimality requirement and baseline-relative repair rules. Here, successful reuse depends on preserving the task's repair conditions in the skill.

The feedback used to build a skill helps explain its contents. The two methods differ in how they turn experience into revisions. SkillEvolver compares successful and failed deployments of a candidate skill, then patches guidance that was missing or misleading. Its independent audit also checks for reliance on training-instance constants [5]. These mechanisms directly target both omitted conditions and instance-specific assumptions. The rules retained in our scheduling and signal-detection skills are consistent with that emphasis: they describe what to check when reusing a procedure.

CoEvoSkills obtains detailed revision feedback from a co-evolving surrogate verifier; the ground-truth oracle returns only an opaque pass/fail signal [7]. Consequently, discovering an omitted condition depends on the surrogate reconstructing it from the task and observed outputs. This offers a plausible explanation for a skill that covers many feasibility checks but misses a condition needed for successful reuse, as in the scheduling example. Both methods generate reusable skill packages; the relevant difference is which failure information becomes persistent guidance. Our interpretation is that feedback coverage and abstraction from the source instance help determine the scope of the resulting skill.

Connecting utility to the evolution process. The results identify useful skills and differences in their applicability; the examples suggest how retained guidance relates to those differences. The next studies test where improvements enter the lifecycle, using parent-to-final comparisons and interventions on feedback, selection, skill content, and verification in SkillRevise, SkillHone, and SkillOpt.

4. IDENTIFYING RETAINED SKILL EFFECTS

4.1. Evaluation Design and Contrasts

We analyze three persistent-skill systems with distinct revision mechanisms. SkillRevise converts execution traces and diagnoses into bounded repair principles [2]. SkillHone accumulates practice history and iteratively reflects on the persistent skill [1]. SkillOpt produces structured bounded edits and uses a validation gate and rejected buffer before adoption [3]. The systems are studied as representative mechanisms, not as directly comparable leaderboard entries.

At the paper level, the evidence is exploratory: individual interventions use frozen inputs and predeclared gates, but the combined studies are designed to identify candidate mechanisms rather than estimate benchmark-wide prevalence. A targeted 24-task SkillsBench

panel tests whether historical outcomes survive fresh deployment. The panel is stratified to expose stability failures and method disagreements; it is not a random sample. Three controlled mechanism cases then compare complete parent and final artifacts under predeclared variations. Within each contrast, we hold the executor model, harness, task instance, budget, and verifier version fixed unless one is the named intervention; exact prompts, hashes, seeds, and stopping decisions are preserved in the artifact package. The task is the primary analysis unit, while repeated runs are deployments of the same task-condition pair.

Four comparisons must remain distinct. Historical-pass to fresh-repeat comparisons test retention of a selected outcome, not advantage over a baseline. Final-to-no-skill comparisons estimate fresh utility, not the effect of revision. Only actual-parent to final comparisons identify a revision effect. Applying that comparison to controlled variants tests bounded transfer, not population-average benefit.

4.2. Retention under Fresh Deployment

This experiment tests whether historically selected outcomes survive when the final artifact is frozen and redeployed. Holding task, skill, and condition fixed does not make execution deterministic. Across the 24-task panel, 12 tasks change success status between their two repeats in at least one condition. Aggregate fresh success is 14/48 without a skill, 16/48 with the SkillRevise final artifact, and 12/48 with the SkillHone final artifact. Both matched-lift intervals cross zero (Table 3).

These results establish an identification failure in the common reporting practice: cached or selected success cannot be read as stable deployment capability. They do not establish that either revision method is ineffective. The panel is purposive, and it lacks each method's actual parent condition. Its proper conclusion is narrower: on this sample, neither final-versus-no-skill comparison establishes a clear average fresh-deployment advantage, and fresh repeats are necessary before a selected score is called retained.

Panel and contrast	Main result	Supported inference and boundary
Panel A: Retention under fresh deployment		
Historical pass $\rightarrow$ fresh repeats	12/24 tasks flip within at least one frozen condition; selected-outcome retention is 47.1% for SkillRevise and 55.0% for SkillHone	A selected pass is not a deployment probability; denominators differ and are not a method ranking.
SkillRevise final $\rightarrow$ no-skill	16/48 vs. 14/48; $+4.2$ pp, 95% CI $[-10.4, 20.8]$	No clear fresh-utility advantage on this targeted panel; not a parent-to-final revision estimate.
SkillHone final $\rightarrow$ no-skill	12/48 vs. 14/48; -4.2 pp, 95% CI $[-20.8, 14.6]$	No clear fresh-utility advantage on this targeted panel; not a parent-to-final revision estimate.
Panel B: Revision effects under controlled variations		
Invoice parent $\rightarrow$ final	$0/12 \rightarrow 12/12$; three new same-contract corpora: $0/9 \rightarrow 9/9$	Stable but narrow same-contract transfer.
Energy parent $\rightarrow$ final	Original task plus 5% parameter variant: $1/6 \rightarrow 5/6$	Directional parameter transfer in a small stochastic case.
SkillOpt SearchQA parent $\rightarrow$ final	Independent held-out $9/24 \rightarrow 17/24$; stable-transfer gate fails	Local held-out gain, not stable general procedure learning.

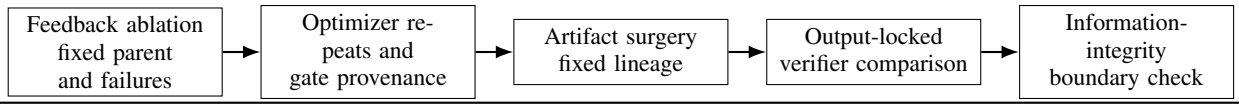
Table 3. Evidence for retained skill effects. Panel A contains 24 targeted tasks, three conditions, and two fresh deployments per condition (144/144 behavior-valid runs). Different historical-retention denominators preclude horizontal method ranking. The no-skill condition calibrates execution and selection variability; final-versus-no-skill does not identify revision effects.

4.3. Revision Effects under Controlled Variations

These mechanism probes test whether the actual parent-to-final revision effect persists under predeclared task changes. In Invoice, the complete final skill repairs all 12 matched deployments and retains the effect on nine deployments drawn from three disjoint corpora that preserve the contract while changing the documents. This is strong evidence of a stable, same-contract revision effect, but not of broad cross-contract generalization.

The Energy case provides weaker but directionally consistent evidence: the complete final artifact improves the original setting and a 5% parameter variant in five of six deployments, compared with one of six for the parent. The sample is too small to support a prevalence or generality claim. In SkillOpt SearchQA, the final artifact improves independent held-out accuracy from 9/24 to 17/24, yet fails the predeclared stability threshold. It therefore supports local held-out benefit rather than stable transferable procedure learning.

Taken together, revised skill artifacts can produce retained behavioral effects, but a selected benchmark pass alone does not identify such an effect, and the effects established so far remain narrow rather than broadly generalizable.



Lifecycle stage	SkillRevise	SkillHone	SkillOpt
Feedback and revision	Diagnosis ablation; revision traces	Feedback-view ablation	Feedback branches; optimizer repeats
Selection and adoption	Revision history	Iterative honing history	Native validation gate; accepted and rejected candidates
Artifact mediation	Not yet matched at the same strength	Invoice and Energy surgery	SearchQA component surgery
Measurement	No output-locked comparison yet	Radar verifier swap	Spreadsheet dual scoring
Information integrity	Artifact forensic	Artifact forensic	Wrong-metadata and oracle controls

Fig. 2. Lifecycle attribution design and current evidence coverage. The upper panel locates each intervention; the lower panel records method coverage. Populated cells do not imply equivalent strength, and missing cells bound cross-method claims.

5. ATTRIBUTING OBSERVED SKILL GAINS

5.1. Lifecycle Interventions

The experiments in this section intervene at different lifecycle stages. They do not estimate one common effect size. Each intervention changes one object while holding its relevant lineage fixed: feedback-view studies hold the parent and failures fixed; optimizer-repeat studies hold the input evidence fixed; gate audits preserve accepted and rejected candidates; artifact surgery holds the surrounding artifact fixed; and verifier counterfactuals hold the output fixed.

Figure 2 makes the current coverage and its gaps explicit. The shared audit logic is that feedback access, candidate revision, validation acceptance, adoption into the persistent artifact, and retained behavioral improvement are distinct events.

5.2. Revision and Selection Pathways

These interventions test whether the observed endpoint depends on the feedback view, optimizer sample, or selection gate. In SkillRevise, the complete three-revision pipeline succeeds on 53/86 tasks, compared with 28/86 when diagnosis is removed. Task-level discordance is 26 gains versus one loss, and 2,326 of 3,648 revision entries explicitly use verifier evidence. This demonstrates that diagnosis changes the selected endpoint and that the revision process is strongly evaluator-facing. It does not, by itself, establish which changes are retained under fresh deployment.

SkillHone separates access to feedback from stable benefit. Detailed sanitized, binary, and redacted-tail feedback obtain first-deployment success on 4/12, 4/12, and 5/12 cases, while four of 18 repeated cells flip. Feedback can trigger revisions, but these data do not support a stable advantage for one feedback view.

SkillOpt further separates candidate production from attribution and adoption. In matched SearchQA optimizer repeats, raw-gold and outcome-only views each produce four applied edits. Yet the preregistered between-condition effect size is only $D = .044 < .20$, despite weak lexical decodability, so a single patch cannot be cleanly attributed to feedback view. Across outcome-only and trajectory-only branches, all 6/6 candidates change and 3/6 pass the gate, but every acceptance is driven by the same Blackwater validation item. In OfficeQA, a selected one-item gain disappears in fresh matched deployment. Thus edit production, gate acceptance, and retained improvement cannot be collapsed into one event.

5.3. Artifact-Level Mediation

This experiment tests whether a feedback-derived component, rather than unrelated edits in the full artifact, causes the behavior after adoption. Let P denote the complete parent, F the complete final artifact, $P + B$ the parent with component B, and $F - B$ the final artifact with that component removed.

Case	Result	Maximum claim
Invoice	$P = 0/12$, $P + B = 12/12$, $F - B = 0/12$, $F = 12/12$	B is necessary and sufficient for the narrow same-contract effect.
Energy	Local mediation in the original task; necessity and sufficiency do not survive the variant	Local component effect, not transferable mediation.
SearchQA	9 repairs, 0 regressions; stable success 7/12, below the strong gate	Directional mediation on part of the held-out set.

Table 4. Artifact-level mediation evidence.

These interventions establish that an adopted artifact component can cause behavior. They do not show that the same mechanism is prevalent across methods. In particular, SkillRevise lacks a matched surgery of equal strength, and the current cases should be read as existence and boundary evidence.

5.4. Measurement Dependence under Fixed Outputs

This counterfactual tests whether a verdict changes when the output is fixed and only the measurement changes. In the Radar family, all 12 reconstructed outputs reproduce their historical original-verifier outcomes. Of six original passes, only one remains a pass under a reference-calibrated strict verifier; the five valid flips arise from specification-implied checks. The same behavior can therefore receive different valid verdicts because the measurement surfaces differ.

Measurement dependence can also operate in the other direction. For identical Spreadsheet workbook outputs, the reported value-based evaluator scores the complete SkillOpt artifact at 10/12,

whereas joint value-and-formula recalculation scores it at 8/12. Representation, cache materialization, or formula inspection can therefore change what the evaluator observes. This dual scoring is not identical in design to the Radar output-locked swap, but it reinforces the narrower point that score is a property of output plus measurement, not behavior alone.

Neither result proves semantic failure, exploitation, or benchmark-wide prevalence. To identify evolution-induced verifier dependence, one would additionally need matched no-skill, parent, and final outputs under both original and calibrated verifiers, and test whether the parent-to-final verifier gap increases.

5.5. Information-Integrity Boundary

This supporting check tests whether restricted answer or oracle information enters and controls the artifact. High-confidence direct-answer or oracle matches are absent from 86 SkillRevise and 85 SkillHone natural artifacts. SkillOpt conditions containing the feedback-derived answer-surface section follow incorrect metadata in 0/15 probes. In contrast, an unsafe hidden-oracle positive control changes probe success from 0/5 to 5/5 and leaves private-oracle or gold markers.

These checks show that the audit can detect direct exposure when it is deliberately introduced, while the natural artifacts do not support direct copying as a universal explanation. They do not prove the absence of implicit answer-following policies or all forms of leakage.

Observed score gains can arise at multiple lifecycle stages: feedback shapes candidate revisions, selection can amplify narrow improvements, adopted artifact components can mediate real behavior, and valid verifiers can disagree on fixed outputs. These mechanisms must be identified separately before a score gain is interpreted as skill learning.

6. FROM REPORTED GAINS TO AUDITABLE CLAIMS

6.1. Evidence Requirements and Claim Boundaries

The audit is not a scalar ladder from *safe* to *unsafe*. Its steps answer independent questions and impose distinct claim boundaries.

Audit step	Required test → maximum supported claim
Validity	Execution and verifier status plus assertion coverage → eligible observation
Retention	Frozen actual-parent/final fresh repeats → retained artifact effect
Behavioral scope	Entity, parameter, input, contract, or family variants → bounded controlled transfer
Evolution pathway	Feedback ablation, optimizer repeats, and gate provenance → pathway dependence or selection mediation
Measurement	Output-locked calibrated verifier comparison → measurement dependence
Information integrity	Provenance, wrong-metadata, and positive control → exploitation or confirmed leakage, when supported

Table 5. Evidence requirements and claim boundaries. The rows answer complementary questions rather than forming a scalar risk ladder.

A selected gain is weaker than a retained artifact effect; a retained effect on one instance is weaker than controlled transfer. Evolution-pathway and measurement claims remain meaningful even when retention is not established. Conversely, verifier-aware language or evaluator dependence must not be relabeled as leakage without restricted-information provenance and behavioral evidence.

6.2. Lifecycle Reporting Protocol

An auditable self-evolution report should preserve the following minimum evidence:

- **Before optimization:** parent artifact, task and split identity, failure pool, environment, and verifier version.
- **During revision:** optimizer-visible traces and feedback, candidate patches, optimizer samples, prompts, and usage.
- **At selection and adoption:** accepted and rejected artifacts, gate items, per-item decisions, and exact adopted artifact.
- **At deployment:** frozen artifact, actual-parent and no-skill conditions, fresh repeats, controlled variants, and output bytes.
- **At scoring:** verifier logs, assertion coverage, reference calibration, and any alternative verifier applied to the same output.
- **For cost:** separate optimizer, executor, verifier, and invalid-attempt ledgers.

The complete artifact appendix should additionally record hashes, commands, seeds, environment revisions, timeouts, and stopping decisions. This protocol turns a post-evolution score into an auditable lineage and limits the paper's language to the strongest matched contrast actually performed.

7. RELATED WORK

7.1. Skill Evolution and Empirical Evaluation

Agent-skill research spans generation, revision, selection, retrieval, organization, and reinforcement learning. SkillsBench and SWE-Skills-Bench compare skill and no-skill utility across diverse or software-engineering tasks [8, 9]. SkillLearnBench evaluates continual skill generation [10], while SkillRevise, SkillHone, SkillOpt, SkillRL, and CoEvoSkills propose different revision or learning loops [1–4, 7]. Recent empirical work broadens the object of study: From Raw Experience to Skill Consumption separates experience generation, extraction, and consumption [11]; How Well Do Agentic Skills Work in the Wild studies realistic skill retrieval [12]; SkillCoach evaluates process-level skill use [13]; and SkillJuror isolates the effect of skill organization [14].

These studies show that outcome-only evaluation is already recognized as incomplete. Our focus is narrower and complementary: when a persistent skill is revised and the post-revision score rises, what evidence identifies a retained artifact effect rather than a selected or measurement-dependent endpoint? We therefore organize evidence by lifecycle contrast rather than by method score.

7.2. Evaluation and Attribution of Agent Improvement

Agent evaluation increasingly examines process quality, sequential adaptation, verifier reliability, and counterfactual behavior. Counterfactual Trace Auditing measures how a provided skill changes an

agent trajectory [15]. SEA-Eval evaluates self-evolving agents beyond isolated episodes [16]. Co-evolving verifier work and grader audits study interactions between the policy and its measurement signal [7, 17]. More broadly, reward hacking and test-set contamination research demonstrates that performance on a proxy or exposed test need not imply the intended capability [18, 19].

We combine these concerns within the lifecycle of persistent skill revision. The central distinction is between retention, behavioral scope, evolution-and-selection pathway, and output-level measurement dependence. Information-integrity checks play a supporting role: they bound direct answer or oracle exposure, but do not convert every verifier-aware adaptation into a leakage claim.

8. LIMITATIONS

Our evidence is mechanism-focused rather than a benchmark-wide prevalence study. The 24-task retention panel is purposive and contains only two fresh deployments per condition; it exposes instability but cannot estimate deployment rates. The controlled Invoice, Energy, SearchQA, Radar, Spreadsheet, and OfficeQA probes use small, heterogeneous task families rather than a fully crossed method-by-intervention design. SkillRevise, for example, lacks matched artifact surgery and output-locked measurement evidence of the same strength, while the strongest verifier swap is concentrated in the Radar family. We therefore do not claim that any observed mechanism is universal or that one method outperforms another.

Several conclusions are also bounded by observability. Output-locked comparisons identify measurement dependence but do not determine which verifier best captures intended behavior without an external specification. Artifact forensics and wrong-metadata controls can rule out simple direct exposure, but not implicit answer-following policies or every form of contamination. Finally, our scope is frozen-agent, feedback-driven systems with persistent external artifacts; parameter-updating self-evolution, multimodal skills, and online environments may exhibit different lifecycle dynamics. Confirmatory evidence will require a shared sampling plan that fully crosses actual parent and final artifacts, fresh repeats, controlled variants, and calibrated verifiers.

9. CONCLUSION

We diagnose where the gains from self-evolving persistent skills reside and how far they extend. All three methods in the standard-task evaluations have positive paired gains in both runs, but task variation separates their utility profiles. SkillEvolver remains positive across representation, instance, and requirement changes; SPARK combines gains on two probes with a decline on instance changes; CoEvoSkills declines on all three. The inspected traces identify decision criteria and correctness checks expressed during successful reuse. Complementary lifecycle interventions show that adopted artifact components can change behavior, selected outcomes can disappear under fresh deployment, and fixed outputs can receive different verdicts under different verifiers. Together, these findings distinguish useful artifacts, their behavioral scope, and the processes that produce or measure their apparent improvement.

A higher benchmark score should therefore be reported only at the strongest level supported by matched evidence: a selected gain, a retained artifact effect, bounded controlled transfer, evolution-pathway dependence, or measurement dependence. We provide a lifecycle audit that links each claim to the required intervention and reporting evidence. Claims of transferable skill learning require frozen parent-to-final improvement that survives fresh deployment and predeclared task variation under valid measurement; weaker evidence should be reported more narrowly.

10. REFERENCES

[1] Zhiwei Li and Yong Hu, "SkillHone: A Harness for Continual Agent Skill Evolution Through Persistent Decision History," 2026.

[2] Yuxuan Liu, Zhaochen Su, Lingyun Xie, Yuhao Zhang, Qing Zong, Jiahe Guo, Zhongwei Xie, Yiyan Ji, Yauwai Yim, Hongyu Luo, Xiyu Ren, Chenyu Ruan, Haoran Li, and Yangqiu Song, "SkillRevise: Improving LLM-Authored Agent Skills via Trace-Conditioned Skill Revision," 2026.

[3] Yifan Yang, Ziyang Gong, Weiquan Huang, Qihao Yang, Ziwei Zhou, Zisu Huang, Yan Li, Xuemei Gao, Qi Dai, Bei Liu, Kai Qiu, Yuqing Yang, Dongdong Chen, Xue Yang, and Chong Luo, "SkillOpt: Executive Strategy for Self-Evolving Agent Skills," 2026.

[4] Peng Xia, Jianwen Chen, Hanyang Wang, Jiaqi Liu, Kaide Zeng, Yu Wang, Siwei Han, Yiyang Zhou, Xujiang Zhao, Haifeng Chen, Zeyu Zheng, Cihang Xie, and Huaxiu Yao, "SkillRL: Evolving Agents via Recursive Skill-Augmented Reinforcement Learning," 2026.

[5] Genrui Zhang, Erle Zhu, Jinfeng Zhou, Caiyan Jia, and Hongning Wang, "SkillEvolver: Skill Learning as a Meta-Skill," 2026.

[6] Yang Zhou, Zihan Dong, Zhenting Wang, Can Jin, Shiyu Zhao, Bangwei Guo, Difei Gu, Linjun Zhang, Mu Zhou, and Dimitris N. Metaxas, "Evidence Over Plans: Online Trajectory Verification for Skill Distillation," 2026.

[7] Hanrong Zhang, Shicheng Fan, Henry Peng Zou, Yankai Chen, Zhenting Wang, Jiayu Zhou, Chengze Li, Wei-Chieh Huang, Yifei Yao, Kening Zheng, Xue Liu, Xiaoxiao Li, and Philip S. Yu, "CoEvoSkills: Self-Evolving Agent Skills via Co-Evolutionary Verification," 2026.

[8] Xiangyi Li, Yimin Liu, Wenbo Chen, Bingran You, Zonglin Di, Yifeng He, Shenghan Zheng, Kyoung Whan Choe, Jiankai Sun, Shuyi Wang, Chujun Tao, Binxu Li, Xuandong Zhao, Hejia Geng, Xiaojun Wu, Junwei Zhou, Xiaokun Chen, Hanwen Xing, Yubo Li, Qunhong Zeng, Di Wang, Yuanli Wang, Roey Ben Chaim, Penghao Jiang, Haotian Shen, Luyang Kong, Xinyi Liu, Runhui Wang, Xuanqing Liu, Jiachen Li, Xin Lan, Yueqian Lin, Wengao Ye, Junwei He, Songlin Li, Yue Zhang, Yipeng Gao, Yijiang Li, Ze Ma, Liqiang Jing, Tianyu Wang, Kaixin Li, Yiqi Xue, Haoran Lyu, Yizhuo He, Yuchen Tian, Shutong Wu, Bowei Wang, Yixuan Gao, Bo Chen, Litong Liu, Sikai Cheng, Jiajun Bao, Shuaicheng Tong, Shuwen Xu, Terry Yue Zhuo, Tinghan Ye, Qi Qi, Miao Li, Longtai Liao, Zelin Tan, Chang Shi, Xilin Tang, Srinath Tankasala, Boqin Yuan, Yaoyao Qian, Jianhong Tu, Chenguang Wang, Yizhou Sun, Wei Wang, Aaron Taylor, Ziyue Yang, Changkun Guan, Zhikang Dong, Xinyu Zhang, Steven Dillmann, Han chung Lee, and Dawn Song, "SkillsBench: Benchmarking How Well Agent Skills Work Across Diverse Tasks," 2026.

[9] Tingxu Han, Yi Zhang, Wei Song, Chunrong Fang, Zhenyu Chen, Youcheng Sun, and Lijie Hu, "SWE-Skills-Bench: Do Agent Skills Actually Help in Real-World Software Engineering?," 2026.

[10] Shanshan Zhong, Yi Lu, Jingjie Ning, Yibing Wan, Lihan Feng, Yuyi Ao, Leonardo F. R. Ribeiro, Markus Dreyer, Sean Ammirati, and Chenyan Xiong, "SkillLearnBench: Benchmarking Continual Learning Methods for Agent Skill Generation on Real-World Tasks," 2026.

[11] Zisu Huang, Jingwen Xu, Yifan Yang, Ziyang Gong, Qihao Yang, Muzhao Tian, Xiaohua Wang, Changze Lv, Xuemei Gao, Qi Dai, Bei Liu, Kai Qiu, Xue Yang, Dongdong Chen, Xiaoqing Zheng, and Chong Luo, "From Raw Experience to Skill Consumption: A Systematic Study of Model-Generated Agent Skills," 2026.

[12] Yujian Liu, Jiabao Ji, Li An, Tommi Jaakkola, Yang Zhang, and Shiyu Chang, "How Well Do Agentic Skills Work in the Wild: Benchmarking LLM Skill Usage in Realistic Settings," 2026.

[13] Jiayin Zhu, Kelong Mao, Yudong Guo, Dengbo He, Sulong Xu, Simiu Gu, and Yutao Yue, "SkillCoach: Self-Evolving Rubrics for Evaluating and Enhancing Agentic Skill-Use," 2026.

[14] Zhiyu Chen, Zihan Guo, Bo Huang, Bingwei Lu, Jianghao Lin, Yuanjian Zhou, and Weinan Zhang, "SkillJuror: Measuring How Agent Skill Organization Changes Runtime Behavior," 2026.

[15] Xiaolin Zhou, Jinbo Liu, Li Li, Ryan A. Rossi, and Xiyang Hu, "Counterfactual Trace Auditing of LLM Agent Skills," 2026.

[16] Sihang Jiang, Lipeng Ma, Zhonghua Hong, Keyi Wang, Zhiyu Lu, Tengfei Wang, Shisong Chen, Jinghao Zhang, Tianjun Pan, Weijia Li, Jiaqing Liang, and Yanghua Xiao, "SEA-Eval: A Benchmark for Evaluating Self-Evolving Agents Beyond Episodic Assessment," 2026.

[17] Xing Zhang, Guanghui Wang, Yanwei Cui, Ziyuan Li, Wei Qiu, Bing Zhu, and Peiyang He, "Who Grades the Grader? Co-Evolving Evaluation Metrics and Skills for Self-Improving LLM Agents," 2026.

[18] Dario Amodei, Chris Olah, Jacob Steinhardt, Paul Christiano, John Schulman, and Dan Mané, "Concrete Problems in AI Safety," 2016.

[19] Yonatan Oren, Nicole Meister, Niladri Chatterji, Faisal Ladhak, and Tatsunori B. Hashimoto, "Proving Test Set Contamination in Black Box Language Models," 2023.